Universidad Andrés Bello Online

Guía de Ejercicios
Unidad 3

Guía de Ejercicios Unidad 3

Ejercicio 1: Análisis Descriptivo de Datos con Pandas y Matplotlib

Enunciado: trabajarás con un conjunto de datos simple sobre las ventas de diferentes productos. Tu tarea es cargar estos datos en un *DataFrame* de Pandas y realizar un análisis exploratorio básico utilizando Matplotlib.

Desarrollo Sugerido:

1. Crea un diccionario en Python que contenga datos de ventas. Por ejemplo:

```
datos = {
'producto': ['Laptop', 'Teclado', 'Monitor', 'Mouse', 'Auriculares'],
'ventas': [150, 200, 120, 350, 180]
}
```

1. Importa las librerías pandas y matplotlib.pyplot y crea un DataFrame de Pandas a partir de tu diccionario.
2. Utiliza Matplotlib para crear un gráfico de barras que muestre las ventas de cada producto.
3. Asegúrate de personalizar el gráfico con un título, etiquetas para los ejes (xlabel y ylabel), y colores a tu elección.
4. Usa plt.show() para visualizar el gráfico.

Solución Esperada:

Esta es una solución genérica, se puede personalizar a criterio del programador.

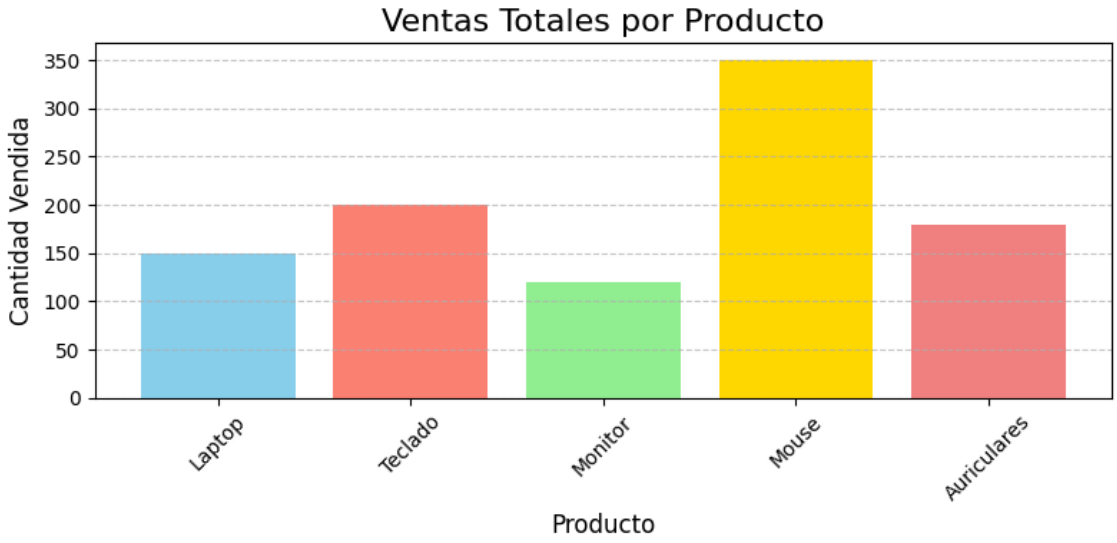
Figura 1

Ejercicio 1 Unidad 3.

```python
import pandas as pd
import matplotlib.pyplot as plt

# 1. Crear el diccionario y el DataFrame
datos = {
    'producto': ['Laptop', 'Teclado', 'Monitor', 'Mouse', 'Auriculares'],
    'ventas': [150, 200, 120, 350, 180]
}
df_ventas = pd.DataFrame(datos)

# 2. Crear y personalizar el gráfico de barras
plt.figure(figsize=(8, 4))
plt.bar(
    df_ventas['producto'],
    df_ventas['ventas'],
    color=['skyblue', 'salmon', 'lightgreen', 'gold', 'lightcoral']
)
plt.title('Ventas Totales por Producto', fontsize=16)
plt.xlabel('Producto', fontsize=12)
plt.ylabel('Cantidad Vendida', fontsize=12)
plt.xticks(rotation=45) # Rotar etiquetas para mejor visibilidad
plt.grid(axis='y', linestyle='--', alpha=0.7)

# 3. Mostrar el gráfico
plt.tight_layout()
plt.show()
```

Figura 2

Gráfico de salida del código de la Figura 1.



Ejercicio 2: Visualización de distribuciones con gráficos de caja y violín

Enunciado: simula datos de salarios para dos departamentos diferentes de una empresa. Tu objetivo es comparar la distribución de salarios entre estos departamentos utilizando un diagrama de caja y bigotes (*boxplot*) y un gráfico de violín.

Desarrollo Sugerido:

1. Importa numpy y matplotlib.pyplot.
2. Genera datos simulados para salarios_marketing y salarios_tecnologia usando np.random.normal().
3. Utiliza plt.subplots() para crear una figura con dos subgráficos, uno al lado del otro.

4. En el primer subgráfico, usa ax1.boxplot() para mostrar la distribución de salarios.
5. En el segundo subgráfico, usa ax2.violinplot() para mostrar la misma distribución de salarios.
6. Añade títulos descriptivos a cada subgráfico y a la figura completa (suptitle).

Solución Esperada:

Esta es una solución genérica, se puede personalizar a criterio del programador.

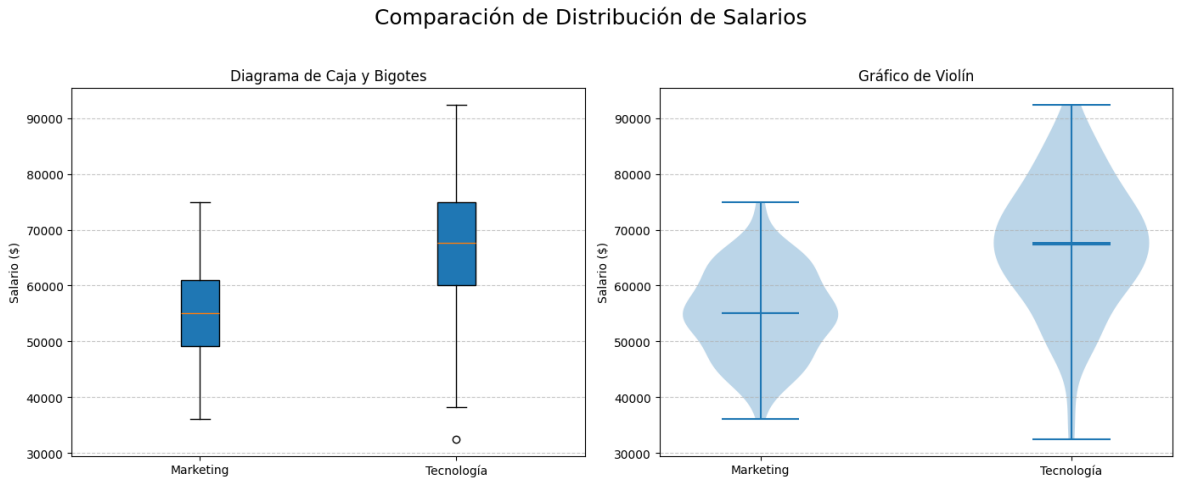
Figura 3

Ejercicio 2 Unidad 3.

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Generar datos simulados
salarios_marketing = np.random.normal(loc=55000, scale=8000, size=100)
salarios_tecnologia = np.random.normal(loc=70000, scale=12000, size=100)
datos_salarios = [salarios_marketing, salarios_tecnologia]
etiquetas = ['Marketing', 'Tecnología']

# 2. Crear subgráficos
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
fig.suptitle('Comparación de Distribución de Salarios', fontsize=18)

# 3. Gráfico de Caja y Bigotes
ax1.boxplot(datos_salarios, tick_labels=etiquetas, patch_artist=True)
ax1.set_title('Diagrama de Caja y Bigotes')
ax1.set_ylabel('Salario ($)')
ax1.grid(axis='y', linestyle='--', alpha=0.7)

# 4. Gráfico de Violín
ax2.violinplot(datos_salarios, showmeans=True, showmedians=True)
ax2.set_title('Gráfico de Violín')
ax2.set_xticks(range(1, len(etiquetas) + 1))
ax2.set_xticklabels(etiquetas)
ax2.set_ylabel('Salario ($)')
ax2.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Ajustar diseño para el título superior
plt.show()
```

Figura 4

Salida del código de la Figura 3



Ejercicio 3: Integración de la API de IA en una función de resumen

Enunciado: desarrolla un programa que tenga una función llamada resumir_texto(texto_largo) que utilice la API de Gemini para generar un resumen. La función debe manejar los errores de conexión o de la API de forma controlada y debe estar protegida para que la clave de API no quede expuesta en el código.

Desarrollo Sugerido:

1. Asegúrate de que la API key de Gemini esté guardada en una variable de entorno.
2. Define una función resumir_texto(texto_largo) que tome un texto como parámetro.
3. Dentro de la función, usa un bloque try...except para configurar la API y llamar al modelo gemini-pro.
4. Formula un prompt claro para pedirle a la IA que resuma el texto.

5. Retorna el texto resumido si la operación es exitosa o un mensaje de error en caso de falla.
6. Llama a la función con un texto largo de tu elección e imprime el resultado.

Solución Esperada:

Esta es una solución genérica, se puede personalizar a criterio del programador.

Figura 5

Ejercicio 3 Unidad 3.

```python
from google import genai
import os

def resumir_texto(texto_largo):
    """
    Función que utiliza la API de Gemini para generar un resumen de un texto.
    Maneja errores de conexión y de la API.
    """
    try:
        api_key = os.environ.get('GEMINI_API_KEY')
        if not api_key:
            raise ValueError("La variable de entorno GEMINI_API_KEY no está configurada.")

        client = genai.Client(api_key=api_key)
        model = 'gemma-3n-e2b-it'

        # El prompt le pide a la IA un resumen
        prompt = f"Resume el siguiente texto en un par de frases: '{texto_largo}'"

        # Generar contenido
        response = client.models.generate_content(model=model, contents=prompt)

        # Devolver el texto del resumen
        return response.text

    except Exception as e:
        return f"Error al generar el resumen: {e}"

# Texto de ejemplo
texto_ejemplo = """
La fotosíntesis es un proceso bioquímico vital que las plantas, algas y algunas bacterias
utilizan para convertir la luz solar, el dióxido de carbono y el agua en energía química,
almacenada en forma de glucosa, y liberar oxígeno como subproducto. Este proceso es
fundamental para la vida en la Tierra, ya que es la principal fuente de oxígeno en la
atmósfera y la base de la mayoría de las cadenas alimenticias.
"""

# Llamar a la función
resumen = resumir_texto(texto_ejemplo)
print("Texto original:")
print(texto_ejemplo)
print("\nResumen generado:")
print(resumen)
```

Salida del modelo

Texto original:
La fotosíntesis es un proceso bioquímico vital que las plantas, algas y algunas bacterias utilizan para convertir la luz solar, el dióxido de carbono y el agua en energía química, almacenada en forma de glucosa, y liberar oxígeno como subproducto. Este proceso es fundamental para la vida en la Tierra, ya que es la principal fuente de oxígeno en la atmósfera y la base de la mayoría de las cadenas alimenticias.

Resumen generado:
La fotosíntesis es un proceso crucial en el que plantas, algas y bacterias transforman la luz solar, el CO2 y el agua en glucosa (energía) y oxígeno. Este proceso es esencial para la vida en la Tierra, ya que proporciona la mayor parte del oxígeno atmosférico y sustenta la mayoría de las cadenas alimentarias.

Ejercicio 4: Generación de Listas con la API de IA en un Bucle

Enunciado: crea un programa interactivo donde el usuario pueda solicitar a la API de Gemini que genere una lista de elementos para una categoría específica. El programa debe continuar pidiendo categorías hasta que el usuario ingrese la palabra "salir".

Desarrollo Sugerido:

1. Asegúrate de tener la configuración de la API de Gemini en tu programa.
2. Inicia un bucle while True para una interacción continua.
3. Dentro del bucle, pide al usuario que ingrese una categoría (input()).
4. Si la entrada del usuario es "salir", usa break para terminar el bucle.
5. Crea un prompt que le pida a la IA que genere una lista de 5 elementos para la categoría ingresada.
6. Usa un bloque try...except para llamar a la API y procesar la respuesta.
7. Imprime la lista generada en un formato claro para el usuario.

Solución Esperada:

Esta es una solución genérica, se puede personalizar a criterio del programador.

Figura 6

Ejercicio 4 Unidad 3.

```
from google import genai
import os

try:
    api_key = os.environ.get('GEMINI_API_KEY')
    if not api_key:
        raise ValueError("La variable de entorno GEMINI_API_KEY no está configurada.")
    client = genai.Client(api_key=api_key)
    model = 'gemma-3n-e2b-it'
except Exception as e:
    print(f"Error de configuración: {e}")
    exit()

print("Generador de listas IA. Ingresa una categoría (o 'salir' para terminar).")

while True:
    categoria = input("\nIngresa una categoría: ")

    if categoria.lower() == 'salir':
        print("¡Hasta la próxima!")
        break

    prompt = f"Genera una lista de 5 elementos para la categoría '{categoria}'. Solo dame la lista."

    try:
        response = client.models.generate_content(model=model, contents=prompt)

        print("\nLista generada:")
        print(response.text)

    except Exception as e:
        print(f"Error al generar la lista: {e}")
```

Salida del modelo

Figura 7

Salida del modelo Ejercicio 4 Unidad 3

```
Generador de listas IA. Ingresa una categoría (o 'salir' para terminar).

Ingresa una categoría:  animales de áfrica

Lista generada:
1. León
2. Elefante africano
3. Chita
4. Hipopótamo
5. Rinoceronte blanco

Ingresa una categoría:
```

Ejercicio 5: Afirmaciones (assert) en un Analizador de Datos

Enunciado: crea una clase AnalizadorDeDatos que tome una lista de números. En su constructor, utiliza assert para garantizar que la lista no esté vacía y que todos sus elementos sean numéricos. Luego, implementa un método para graficar la distribución de los datos usando un histograma de Matplotlib.

Desarrollo Sugerido:

1. Define la clase AnalizadorDeDatos con un constructor que reciba datos.
2. En el constructor, usa assert para verificar si la lista de datos no está vacía.
3. Usa otro assert dentro de una comprensión de lista para verificar que todos los elementos sean int o float.
4. Implementa un método graficar_histograma(self) que cree y muestre un histograma de los datos.

5. Crea un objeto con datos válidos para verificar su correcto funcionamiento.
6. Usa un bloque try...except para intentar crear un objeto con datos inválidos (ej. una lista vacía o una lista con una cadena de texto) y captura el AssertionError.

Solución Esperada:

Esta es una solución genérica, se puede personalizar a criterio del programador.

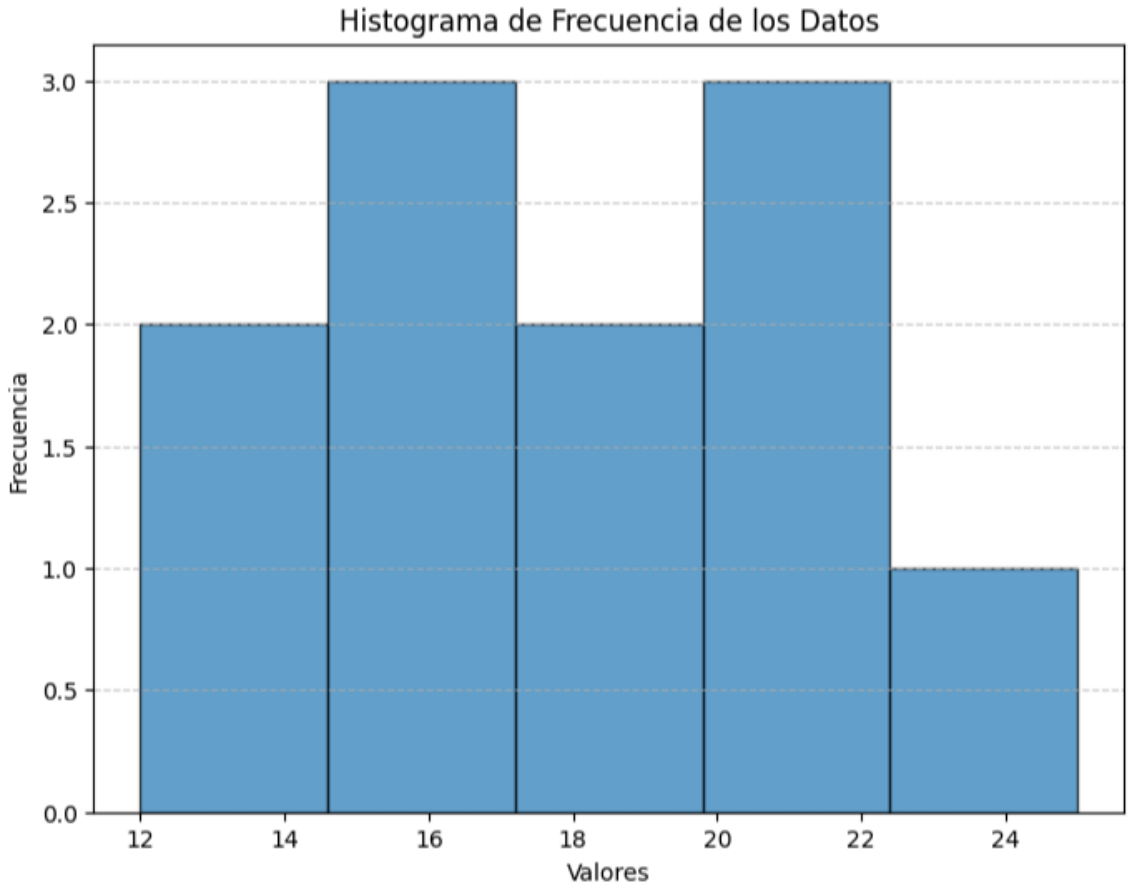
Figura 8

Ejercicio 5 Unidad 3.

```python
import matplotlib.pyplot as plt
import numpy as np

class AnalizadorDeDatos:
    def __init__(self, datos):
        # 1. Usar assert para validaciones
        assert isinstance(datos, list) and len(datos) > 0, "Error: Los datos deben ser una lista no vacía."
        assert all(isinstance(x, (int, float)) for x in datos), "Error: Todos los elementos de la lista deben ser numéricos."
        self.datos = datos

    def graficar_histograma(self):
        plt.figure(figsize=(8, 6))
        plt.hist(self.datos, bins='auto', edgecolor='black', alpha=0.7)
        plt.title('Histograma de Frecuencia de los Datos')
        plt.xlabel('Valores')
        plt.ylabel('Frecuencia')
        plt.grid(axis='y', linestyle='--', alpha=0.7)
        plt.show()

# Escenario exitoso
try:
    datos_validos = [12, 15, 20, 22, 18, 15, 25, 20, 19, 15, 12]
    analizador_ok = AnalizadorDeDatos(datos_validos)
    analizador_ok.graficar_histograma()
except AssertionError as e:
    print(f"Error: {e}")

# Escenario con error (lista vacía)
try:
    datos_vacios = []
    analizador_error_vacio = AnalizadorDeDatos(datos_vacios)
except AssertionError as e:
    print(f"\nError capturado: {e}")

# Escenario con error (datos no numéricos)
try:
    datos_no_numericos = [1, 2, 'tres', 4]
    analizador_error_tipo = AnalizadorDeDatos(datos_no_numericos)
except AssertionError as e:
    print(f"\nError capturado: {e}")
```

Figura 9

Salida del código de la Figura 8.




```
Error capturado: Error: Los datos deben ser una lista no vacía.

Error capturado: Error: Todos los elementos de la lista deben ser numéricos.
```